

UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA
FACULTAD DE INGENIERÍA

MANUAL TÉCNICO DE DESARROLLO E INFRAESTRUCTURA

Sistema de Gestión de Recursos del Centro de Cómputo (SGRC)

Programa Educativo: Ingeniería en Ciencias de la Computación

Asignatura: Bases de Datos & Desarrollo Basado en Plataformas

Grupo: 6cc2

Equipo de Desarrollo:

Nicolás Paúl Nevárez Loera (Matrícula: 367886)

Jonathan Gandara Salazar (Matrícula: 374357)

Samuel Eduardo García Gómez (Matrícula: 367651)

1. Arquitectura General del Sistema

El Sistema de Gestión de Recursos del Centro de Cómputo (SGRC) v1.5 implementa una arquitectura full-stack desacoplada, distribuida en tres capas y orquestada completamente bajo contenedores Docker. Este sistema está diseñado para la administración concurrente y en tiempo real de cubículos de estudio, solicitudes digitales de préstamo de equipo e interacciones del módulo de tutorías académicas.

1.1 Capa de Presentación (Frontend)

La interfaz gráfica de usuario está construida sobre **React 18** utilizando el empaquetador moderno **Vite** y administrada mediante el gestor de dependencias `pnpm`. Los estilos visuales se rigen bajo **Tailwind CSS** e implementan elementos tipográficos institucionales de la UACH (como las fuentes Praxis y Alverata) y un sistema centralizado de íconos SVG vectoriales propios. Con el objetivo de optimizar la velocidad de renderizado y mantener un consumo mínimo de memoria, el estado de la aplicación se maneja localmente mediante hooks nativos de React (`useState`, `useEffect`) y peticiones asíncronas vía `fetch`, descartando la complejidad innecesaria de librerías globales como Redux o Zustand.

1.2 Capa de Lógica de Negocio (Backend)

La API de servicios está desarrollada en **Spring Boot 3.x** corriendo bajo **Java 21**. Aplica el patrón de diseño arquitectónico MVC modificado para la exposición de servicios web REST puros. La seguridad perimetral y el control de accesos basado en roles (RBAC) se gestionan mediante **Spring Security** configurado de manera apátrida (Stateless), validando las cabeceras mediante tokens criptográficos **JWT (JSON Web Token)**. Para las comunicaciones reactivas y la arquitectura orientada a mensajes/eventos de reservación, el backend integra el broker de mensajería **RabbitMQ** coordinado por **Apache Camel**, además de un servidor de **WebSockets** utilizando el protocolo STOMP sobre SockJS** para notificaciones activas en sesión.

1.3 Capa de Persistencia

La persistencia relacional recae sobre el motor **MySQL 8.0**. El mapeo objeto-relacional (ORM) se realiza con Spring Data JPA implementando Hibernate de manera nativa. El esquema de base de datos se mantiene estructurado bajo las reglas de la Tercera Forma Normal (3FN), garantizando la integridad referencial y transaccional mediante llaves foráneas e índices optimizados.

2. Despliegue e Infraestructura con Docker Compose

El entorno completo está automatizado mediante un archivo `docker-compose.yml` que levanta de forma aislada los contenedores de la base de datos (MySQL 8.0) y el servidor de mensajería (RabbitMQ), exponiendo los puertos locales necesarios para la interconexión con el backend (puerto 3000) y el frontend (puerto 5173 con proxy inverso hacia `/api`).

2.1 Estrategia de Sincronización del Esquema

El archivo de configuración de Spring backend (`application.properties`) se encuentra parametrizado bajo la propiedad:

```
spring.jpa.hibernate.ddl-auto=update
```

Esta configuración permite que el ORM Hibernate modifique o actualice la estructura de las tablas en tiempo de ejecución en caso de detectar discrepancias con las clases de entidad Java. No obstante, para el despliegue inicial y las pruebas automatizadas de volumen, el script `init.sql` funge como la fuente de verdad inicial, encargándose de estructurar el esquema físico base e inyectar el catálogo maestro junto con un volumen masivo de datos semilla.

3. Diseño Estricto de la Base de Datos

En concordancia con el alcance funcional de la versión 1.5, se ejecutó un recorte de alcance (*Scope Management*) eliminando las tablas muertas `loan` e `import_log` . De igual forma, la persistencia de la tabla `notification` fue descartada debido a que el módulo de avisos se suple en tiempo real mediante la inmediatez del protocolo WebSockets, dejando las notificaciones persistidas como un trabajo futuro. El modelo resultante consta de **9 tablas activas e interconectadas**.

3.1 Diccionario del Esquema Físico (Las 9 Tablas de la Base de Datos)

Nombre de Tabla	Descripción Técnica y Relación Semántica
<code>faculty</code>	Entidad raíz que representa la institución (Facultad de Ingeniería). Permite la escalabilidad futura para despliegues multi-facultad.
<code>user</code>	Almacena las cuentas de usuario de la facultad. Contiene los campos booleanos <code>is_tutor</code> , <code>is_blocked</code> y <code>must_change_password</code> para auditorías de seguridad y privilegios de horario. Sus roles están restringidos por un ENUM ('STUDENT', 'TEACHER', 'ADMIN', 'RECEPTOR').
<code>cubicle</code>	Representa los espacios físicos de estudio dentro de la biblioteca central. Incluye la columna indexada <code>qr_token</code> (VARCHAR 100 UNIQUE) que almacena el UUID estático que lee el iPad receptor en la puerta de entrada.
<code>reservation</code>	Entidad transaccional que mapea el ciclo de vida de los apartados de cubículos. Su estado es controlado por un ENUM ('PENDING', 'APPROVED', 'ACTIVE', 'COMPLETED', 'CANCELLED').
<code>equipment_type</code>	Catálogo maestro del inventario de materiales prestables en el Centro de Cómputo (Laptops, proyectores, marcadores, etc.). Controla los stocks mediante restricciones CHECK para evitar inventarios negativos.
<code>equipment_rental_request</code>	Cabecera transaccional que registra la fecha y el estado de una solicitud digital generada por un alumno desde el frontend móvil.
<code>equipment_rental_request_item</code>	Tabla pivote de desglose (N:M) que asocia múltiples tipos de equipos y cantidades específicas a una única solicitud de renta digital.
<code>penalty</code>	Registro histórico de inhabilitaciones aplicadas a los alumnos. Mapea el tipo de infracción mediante un ENUM ('DISORDER', 'NO_SHOW', 'LATE_CANCELLATION'), registrando fechas exactas de inicio y fin del bloqueo de cuenta.
<code>tutoring_*</code> (4 tablas)	Módulo completo que normaliza las asesorías académicas extratematizadas. Se divide en: <code>tutoring_professor</code> (número de empleado y datos del docente), <code>tutoring_subject</code> (materias registradas), <code>tutoring_offering</code> (bloques de disponibilidad del profesor almacenados en formato de arreglos de texto JSON), y <code>tutoring_request</code> (solicitudes de emparejamiento alumno-maestro).

4. Algoritmos de Servidor y Flujos Críticos

4.1 Scheduler Dinámico Basado en Reservaciones individuales

Para optimizar drásticamente el rendimiento del microprocesador en el servidor, se descartó el uso de un *Cron Job* tradicional que realice barridos de lectura masiva sobre la base de datos cada minuto. En su lugar, el backend implementa un **Scheduler Dinámico Asíncrono**. En el instante exacto en que una reservación de cubículo transiciona al estado `APPROVED`, la capa de servicios de Spring Boot inicializa e inyecta un hilo de tarea programada (Task) único en el planificador interno de Java. Este hilo está calculado con un temporizador de cuenta regresiva que se "despierta" con precisión matemática al `**minuto 15 posterior a la hora de inicio pactada ( start_time )**`.

Al activarse, el hilo evalúa de forma atómica el estado actual de esa reservación específica en la base de datos. Si el registro continúa en estado `APPROVED` (lo que significa que el alumno no se presentó a escanear el QR en el cubículo), el script ejecuta de forma autónoma dos acciones transaccionales:

1. Actualiza el estado de la reserva a `CANCELLED`, liberando instantáneamente la disponibilidad del espacio para el catálogo de los demás estudiantes.
2. Inserta un registro de castigo automático en la tabla `penalty` con el tipo `NO_SHOW`, actualizando la bandera `is_blocked = 1` en la entidad del usuario infractor para inhabilitar su cuenta de forma inmediata.

4.2 Motor de Exportación y Compilación PDF para Órdenes de Equipo

Dado que la tabla transaccional de entregas físicas físicas (`loan`) fue removida, el sistema delega la responsabilidad del vale físico al estudiante mediante un módulo de impresión digital. La capa de servicios del backend incluye un componente que extrae la información de la cabecera `equipment_rental_request` y el desglose de sus respectivos ítems. Mediante la compilación de plantillas en tiempo de ejecución, el sistema genera y formatea un documento binario en formato **PDF institucional**, el cual viaja de regreso al frontend de React como un stream de datos descargable. Este PDF funge como el comprobante oficial o "vale" que el alumno presenta impreso o desde su celular en la ventanilla de atención del Centro de Cómputo.

5. Deuda Técnica y Consideraciones de Diseño Conocidas

En cumplimiento con las buenas prácticas de ingeniería de software y con fines de transparencia en la evaluación académica, se documenta el estado de deuda técnica del proyecto al cierre de esta iteración:

- **Ausencia de Rutas Protegidas en Frontend (ProtectedRoute):** React Router v6 no tiene implementados los componentes de enrutamiento guardián (Guards) del lado del cliente. Un usuario malintencionado puede forzar la navegación web hacia URLs administrativas como `/admin`

ingresando la ruta directa en el navegador. No obstante, la seguridad está garantizada debido a que el Backend intercepta todas las peticiones HTTP y bloquea cualquier acción devolviendo códigos `401 Unauthorized` o `403 Forbidden` si el JWT no posee las firmas del rol ADMIN.

- **Falta de Mecanismo Refresh Token:** El sistema de autenticación emite un JSON Web Token con una caducidad estricta y cableada de 24 horas. Al expirar este periodo, las peticiones HTTP del frontend fallan silenciosamente, dejando la aplicación en un estado de interfaz inerte. El frontend carece de un interceptor de Axios/Fetch que capture el error 401 para redirigir automáticamente al usuario a la pantalla de Login.
- **Evaluación de `must_change_password` Omisa:** Aunque la columna `must_change_password` se encuentra correctamente declarada en la tabla ``user`` e inyectada como un claim dentro del cuerpo del JWT para el análisis del cliente, la lógica de Spring Security en el backend no impide que un usuario consuma servicios core utilizando la contraseña genérica por defecto; la redirección al formulario de cambio de contraseña está delegada únicamente al comportamiento del frontend en React.
- **Código Huérfano de Notificaciones:** El módulo de mensajería asíncrona en Spring Boot conserva clases de entidad Java, repositorios JPA y rutinas de escritura dentro del planificador de tareas para escribir notificaciones. Al removerse la tabla física del script `init.sql`, este bloque operativo representa código huérfano sin controladores REST expuestos ni mapeo en las interfaces de usuario.
- **Falta de Paneles de Gestión en Frontend:** Las controladores para la administración cruda de usuarios (`UserController`) y la consulta o levantamiento de sanciones (`PenaltyRepository`) existen en su totalidad dentro del servidor Java; sin embargo, no se desarrollaron las pantallas gráficas correspondientes en la interfaz de React, requiriendo que estas operaciones administrativas se realicen de forma manual mediante consola SQL por el momento.
- **Llaves Secretas en Texto Plano:** La firma criptográfica secreta (Secret Key) utilizada para el firmado de los JWT se encuentra codificada directamente en texto plano dentro del archivo de configuración `application.properties`, en lugar de ser extraída de forma segura mediante una variable de entorno del sistema operativo host.